

عدد اسمیت

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت

تابعی به نام `is_prime` بنویسید که یک عدد صحیح مثبت را دریافت کرده، اگر عدد اول بود 1 و در غیر این صورت 0 را بازگرداند. سپس تابعی به نام `is_smith` بنویسید که بررسی کند آیا یک عدد، «عدد اسمیت» است یا خیر. **تعریف:** عدد اسمیت یک عدد مرکب (غیراول) است که مجموع ارقام آن با مجموع ارقام عوامل اول آن (با احتساب تکرار) برابر باشد.

در نهایت برنامه‌ای بنویسید که یک عدد صحیح مثبت n را از کاربر گرفته و تمام اعداد اسمیت کوچک‌تر یا مساوی n را چاپ کند.

ورودی

ورودی تنها شامل یک عدد صحیح مثبت n است.

$$1 \leq N \leq 10000$$

خروجی

اعداد اسمیت کوچک‌تر یا مساوی n که در هر خط یکی از آن‌ها چاپ می‌شود.

مثال

ورودی نمونه ۱

خروجی نمونه ۱

4

(توضیح: مجموع ارقام عدد ۴ برابر ۴ است. عوامل اول ۴ عبارتند از ۲ و ۲. مجموع ارقام این عوامل برابر است با $2 + 2 = 4$. چون ۴ یک عدد مرکب است و مجموع ارقامش با مجموع ارقام عوامل اولش برابر است، پس عدد اسمیت است.)

ورودی نمونه ۲

25

خروجی نمونه ۲

4

22

(توضیح: برای عدد ۲۲؛ مجموع ارقام: $2 + 2 = 4$. عوامل اول ۲۲ عبارتند از ۲ و ۱۱. مجموع ارقام عوامل اول: $2 + (1 + 1) = 4$. چون مجموع ارقام عدد با مجموع ارقام عوامل اول برابر است، ۲۲ نیز یک عدد اسمیت محسوب می‌شود.)

عدد بسیار مرکب

- محدودیت زمان: ۲ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت

تابعی به نام `count_divisors` بنویسید که یک عدد صحیح N را گرفته و تعداد مقسوم‌علیه‌های آن را برگرداند. سپس تابعی به نام `is_highly_composite` بنویسید که یک عدد N را دریافت کند و اگر «عدد بسیار مرکب» بود 1 و در غیر این صورت 0 را بازگرداند. **تعریف:** عدد N «بسیار مرکب» است اگر تعداد مقسوم‌علیه‌های آن از تعداد مقسوم‌علیه‌های تمام اعداد مثبت کوچک‌تر از خودش بیشتر باشد.

در نهایت برنامه‌ای بنویسید که عدد N را از کاربر بگیرد و تمام اعداد بسیار مرکب کوچک‌تر یا مساوی N را چاپ کند.

ورودی

ورودی شامل یک عدد صحیح مثبت N است.

$$1 \leq N \leq 1000$$

خروجی

اعداد بسیار مرکب کوچک‌تر یا مساوی N که در هر خط یکی از آن‌ها چاپ می‌شود.

مثال

ورودی نمونه ۱

خروجی نمونه ۱

1
2
4

(توضیح:

- برای ۱:۱ مقسوم‌علیه دارد.
- برای ۲:۲ مقسوم‌علیه دارد (بیشتر از ۱).
- برای ۲:۳ مقسوم‌علیه دارد (مساوی ۲، پس بسیار مرکب نیست).
- برای ۳:۴ مقسوم‌علیه دارد (بیشتر از ۲ و ۳)، پس بسیار مرکب است.

ورودی نمونه ۲

10

خروجی نمونه ۲

1
2
4
6

(توضیح: عدد ۶ دارای ۴ مقسوم‌علیه است که از تعداد مقسوم‌علیه‌های اعداد ۱ تا ۵ بیشتر است.)

اعداد غایب

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت

تابعی به نام `find_missing_elements` بنویسید که یک لیست از اعداد صحیح (که همگی در بازه‌ی ۱ تا N هستند و N برابر با بزرگ‌ترین عدد موجود در لیست است) را دریافت کرده و لیست جدیدی شامل تمام اعداد این بازه که در لیست ورودی «غایب» هستند را برگرداند.

سپس تابعی به نام `sort_list` (بدون استفاده از متد داخلی `sort()`) بنویسید که لیست اعداد غایب را به صورت صعودی مرتب کند.

در نهایت برنامه‌ای بنویسید که اعداد لیست را **یکی‌یکی** از کاربر بگیرد (تا زمانی که عدد ۱- وارد شود). در انتها، با توجه به بزرگ‌ترین عدد واردشده (به‌عنوان N)، اعداد غایب در بازه‌ی ۱ تا N را پیدا کرده و به صورت مرتب شده چاپ کند.

ورودی

اعداد لیست (هر عدد در یک خط)، تا زمانی که عدد ۱- وارد شود.

$$1 \leq N \leq 1000$$

خروجی

اعداد غایب در بازه ۱ تا N که هر کدام در یک خط چاپ می‌شوند.

مثال

ورودی نمونه ۱

1
3

5
-1

خروجی نمونه ۱

2
4

(توضیح: بزرگ‌ترین عدد ورودی ۵ است؛ پس بازه‌ی ما ۱ تا ۵ است. ۱، ۳ و ۵ در لیست وجود دارند، پس ۲ و ۴ غایب هستند.)

ورودی نمونه ۲

6
1
4
3
-1

خروجی نمونه ۲

2
5

(توضیح: بزرگ‌ترین عدد ورودی ۶ است؛ بازه‌ی ما ۱ تا ۶ است. اعداد موجود در این بازه عبارتند از {۱، ۳، ۴، ۶}. اعداد غایب ۲ و ۵ هستند.)

ورودی نمونه ۳

1
3
2
5

خروجی نمونه 3

(توضیح: هیچ خروجی ندارد چون تمام اعداد ۱ تا ۵ موجود هستند. اگر لیست غایب خالی بود، برنامه نباید چیزی چاپ کند.)